H

HireLens

AI Job Description Analyzer built with LangGraph

Submitted by Anushay Ayat

Program Artificial Intelligence and Data Science, SMIT

Python | LangGraph | Groq API | FastAPI | Pydantic | Firebase Auth | Vercel

Abstract

HireLens is a web application that reads a job description and turns it into a structured hiring report. The backend uses a LangGraph workflow with four nodes: information extraction, job classification, skill analysis and hiring report generation. Each node calls a large language model through the Groq API and passes a shared state to the next node. The frontend is written in plain HTML, CSS and JavaScript in a dark futuristic style, with a login page powered by Firebase Authentication. Beyond the required report, the application offers resume matching, job comparison, a history of past reports and export to JSON and PDF. It is deployed on Vercel.

1. Introduction

Job descriptions are long and unstructured. A recruiter or a candidate must read the whole posting to find the job title, the experience required, the skills that matter and the type of employment, and important details are easy to miss.

Large language models are good at reading unstructured text and returning structured answers. LangGraph lets a developer split one large AI task into small steps and connect them as a graph. HireLens combines both ideas: every step does one job, and the final output is a clean report.

Problem statement

Given a job description written as free text, produce a report with the job title, company, required experience, required skills, job category, employment type, technical skills, soft skills and the three most important skills. The system must handle postings of different styles and lengths, and must not crash on empty or meaningless input.

2. Objectives

- Build an AI assistant that converts a job description into a structured hiring report.
- Implement all four LangGraph nodes and pass state between them.
- Display the final report clearly, with extracted skills shown as tags.
- Handle empty or invalid input gracefully.
- Write clean, modular code.
- Add a web interface, JSON export, user login and deployment.

3. System design

Workflow

The workflow follows the assignment specification. Select a node to see what it does.

1 **Information extraction**

2 **Job classification**

3 **Skill analysis**

4 **Hiring report**

Reads the job description and extracts the job title, company name, required experience and required skills. Missing values are returned as "Not specified".

All nodes read from and write to one shared state object. LangGraph merges the output of each node into that state, so later nodes can use the work of earlier ones.

Architecture

Layer	Responsibility
Browser	HTML, CSS and JavaScript interface, login page, report display and exports.
FastAPI service	Endpoints /api/analyze, /api/match and /api/health, plus input validation.
LangGraph pipeline	Runs the four nodes in order and carries the shared state.
Groq LLM	Returns a JSON answer for each node.

Project structure

- **api/index.py**: schemas, the four nodes, the graph and the routes.
- **public/**: index.html, style.css, app.js and firebase-config.js.
- **vercel.json, requirements.txt, .env.example, README.md**: setup and deployment files.

4. Technology stack

Technology	Purpose
Python	Main backend language.
LangGraph	Defines the four nodes and the order they run in.
Groq API (Llama 3.3 70B)	Fast language model inference.
FastAPI	REST API.
Pydantic	Data models and validation of every node output.
HTML, CSS, JavaScript	Frontend without a framework.
Firebase Authentication	Email and Google sign-in.
Vercel	Hosting of the frontend and the API.

5. Implementation

State and nodes

A typed dictionary holds the state: the job description, the extracted information, the classification, the skill analysis and the final report. Each node is a small Python function that receives the state and returns only the keys it adds. This keeps every node easy to read, test and replace.

Structured output with Pydantic

Each node asks the model to reply in JSON only. The reply is parsed and validated with a Pydantic model, so the rest of the program works with predictable fields and default values instead of free text.

Input validation and errors

Before the graph runs, the API checks the input. Empty text returns a message asking the user to paste a job description. Text that is too short returns a message asking for more detail. Input is limited to 8,000 characters. If the model service fails, the API returns a clear error and the frontend shows it next to the input instead of crashing.

Resume matching

A separate endpoint compares a resume with the job description and returns a match score, matched skills, missing skills and one sentence of advice. It is kept outside the four-node workflow so the required pipeline stays exactly as specified.

6. Frontend

The interface uses a dark futuristic style: a deep navy background, cyan and violet accents, glass-like panels and an animated particle network.

Screen	What it shows
Login	A pulsing four-node pipeline that previews the workflow, next to the sign-in form.
Analyze	Text area, run, sample and clear buttons, and four progress steps that follow the pipeline.
Report	Title, company, experience, category, employment type, skill tags, top 3 skill bars, interview questions, JSON and PDF export.
Resume match	Score ring with matched and missing skills.
Compare	Two job descriptions side by side with shared and unique skills.
History	The last 20 reports of the signed-in user.

The layout is responsive, shows keyboard focus clearly and respects the reduced-motion setting of the user's device.

7. Authentication

Firebase Authentication provides email and password sign-up, sign-in with Google and sign-out. Firebase error codes are translated into plain messages such as "Email or password is wrong". A guest option lets people try the application without an account. History is stored separately for each user.

8. Sample run

Input

We are hiring an AI Engineer at ABC Technologies (full-time, on-site). You have 2-4 years of experience with Python, FastAPI, LangGraph and SQL. Strong communication, teamwork and problem solving are a must.

Hiring report

Job title: AI Engineer
Company: ABC Technologies
Experience: 2-4 years
Category: Artificial Intelligence
Employment: Full-time
Technical: Python, FastAPI, LangGraph, SQL
Soft: Communication, Teamwork, Problem solving
Top 3: Python, LangGraph, FastAPI

The result matches the example output in the assignment brief. The wording of the summary and the interview questions can vary slightly between runs because they are generated by the model.

9. Deployment

1. Push the project to a GitHub repository.
2. Import the repository into Vercel. It serves the public folder as the website and runs api/index.py as a Python function.
3. Add GROQ_API_KEY in the Vercel environment variables so the key never appears in the code.
4. Enable Email/Password and Google in Firebase Authentication, paste the web configuration into firebase-config.js and add the Vercel domain to the authorized domains.

10. Challenges

- **Reliable structured output:** models sometimes add extra text. JSON mode and Pydantic validation solved this.
- **State passing:** keeping each node focused while still using earlier results needed a clear state design.
- **Bad input:** empty, very short or irrelevant text had to be rejected early with useful messages.
- **Secrets:** the API key stays on the server and is never sent to the browser.

11. Future scope

- Upload job descriptions as PDF or DOCX files.
- Store history in Firestore so it follows the user across devices.
- Rank many resumes against one job.
- Add salary range and market demand insights.
- Support job descriptions in more languages.

12. Conclusion

HireLens meets the requirements of the assignment: four LangGraph nodes, state passed between them, a structured hiring report, graceful handling of invalid input and modular code. It goes further with a complete interface, login, resume matching, comparison, history and export, and it is ready to deploy on Vercel. The project shows that splitting an AI task into small graph nodes produces results that are easy to understand, test and extend.

References: LangGraph, Groq, FastAPI, Pydantic, Firebase Authentication and Vercel documentation.